PLATFORM ENGINEERING · AI AUTOMATION

김연수

Yeonsoo Kim



AI 서비스를 구현하고, 운영을 자동화합니다.

쿠버네티스와 AWS 환경에서 성능을 개선하고 운영을 자동화하는 플랫폼 엔지니어입니다. 오픈소스 기여와 AI 서비스 인턴십, 백엔드 개발 경험을 바탕으로 서비스 구현부터 배포와 운영까지 연결합니다.

인턴십에서는 고객의 데이터 갱신 지연을 AX 과제로 발굴하고, 자율 실행 에이전트와 셀프서비스 플랫폼으로 업무를 자동화했습니다. 고객사의 평균 솔루션 도입 기간을 6주에서 4주로 단축하는 데 기여했습니다.

Argo CD Agent의 반복 조회 성능을 개선했고, Kubernetes Operator와 AI 로그 분석으로 장애 대응 업무를 자동화했습니다. 연구에서는 모델별 NPU 코어 배분을 조정해 제한된 자원의 활용도를 높였습니다.

홈페이지 ↗ GitHub ↗ LinkedIn ↗ 기술 블로그 ↗

대표 경험

채널코퍼레이션 인턴 02
고객용 플랫폼 구축과 사내 AI 에이전트 개선

Argo CD Agent 04
쿠버네티스 리소스 조회를 로컬 캐시로 개선

연구 06
VLM·객체 탐지 모델의 SLO 스케줄링

실시간 결제 서비스 07
AWS 인프라 설계 및 CI/CD 배포 자동화

Kubernetes Operator 12
배포 상태 점검과 로그 조회 기반 운영 자동화

프로젝트 13
햇제 · 뮤사이

학력 · 자격증 · 수상 · 개발 활동 14

채널코퍼레이션 인턴

2026.03 — 2026.06 Forward Deployed Engineer (FDE)

AX(AI Transformation) · 사내 업무 자동화

System Integration | API | Nginx

자사 AI 애플리케이션의 고객 요청 해결률을 20%에서 80%로 향상했습니다. 고객용 셀프서비스 플랫폼을 구축해 고객사의 평균 솔루션 도입 기간을 6주에서 4주로 단축하는 데 기여했습니다.

고객용 플랫폼 구축

고객사의 데이터 변경 요청이 잦아 직원이 전처리와 지식베이스 갱신을 반복해야 했습니다. 고객이 직접 최신 데이터를 반영하는 셀프서비스 환경을 제안했습니다.

AS-IS

담당자에게 데이터 갱신 요청

데이터 관리 기능은 사내 VPN이 필요한 직원용 플랫폼에 있었습니다. 고객에게 VPN 계정을 발급하는 방안은 팀 논의 결과 보안상 어려웠습니다.

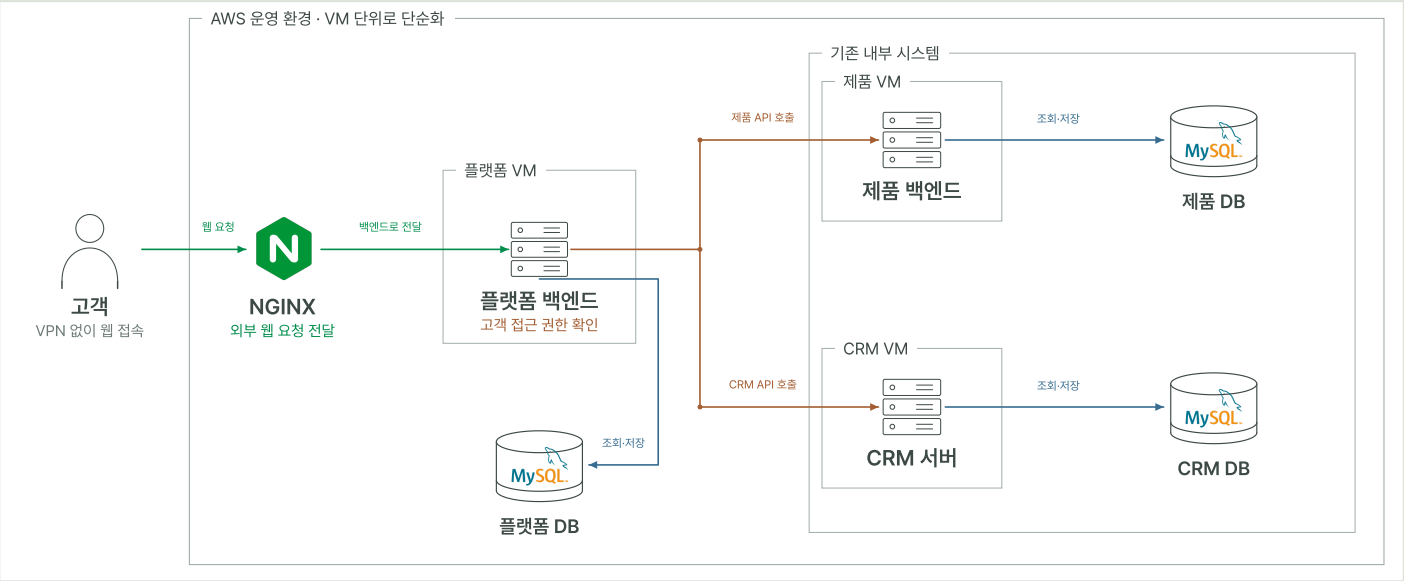
TO-BE

고객이 직접 데이터 등록·갱신

필요한 API를 선별해 Nginx와 백엔드로 연결했습니다. 에이전트 루프를 설계해 고객 요청에 맞는 전처리와 지식베이스 갱신을 수행하도록 했습니다.

- 백엔드에서 고객 인증과 권한 확인
- 기존 ETL 파이프라인을 통해 지식베이스 최신화

내부 API 연동 구조



내부 API 연동과 데이터 최신화

Nginx는 외부 웹 요청을 고객용 백엔드로 전달하고, 백엔드는 고객의 접근 권한을 확인한 뒤 허용된 내부 API를 호출하도록 구성했습니다. 이를 통해 요청 전달 설정을 업무 로직과 분리해 관리했습니다.

플랫폼에서 전처리한 데이터를 기존 ETL 파이프라인을 통해 지식베이스에 반영해, 제품의 AI 에이전트가 최신 데이터를 참조하도록 연결했습니다.

결과 고객 사용 교육까지 맡아 고객사의 평균 솔루션 도입 기간을 6주에서 4주로 단축하는 데 기여했습니다. 플랫폼의 기능을 MCP로 전사에 공유해 고객사별 기능 등록 절차를 표준화했습니다.

AI 에이전트 파일 입력 구조 설계

AI 에이전트에 어떤 정보를 언제 전달할지 설계했습니다. 파일 저장과 모델이 본문을 읽는 시점을 분리해 반복 입력을 줄였습니다.

AS-IS

파일 본문 전체를 매번 전달

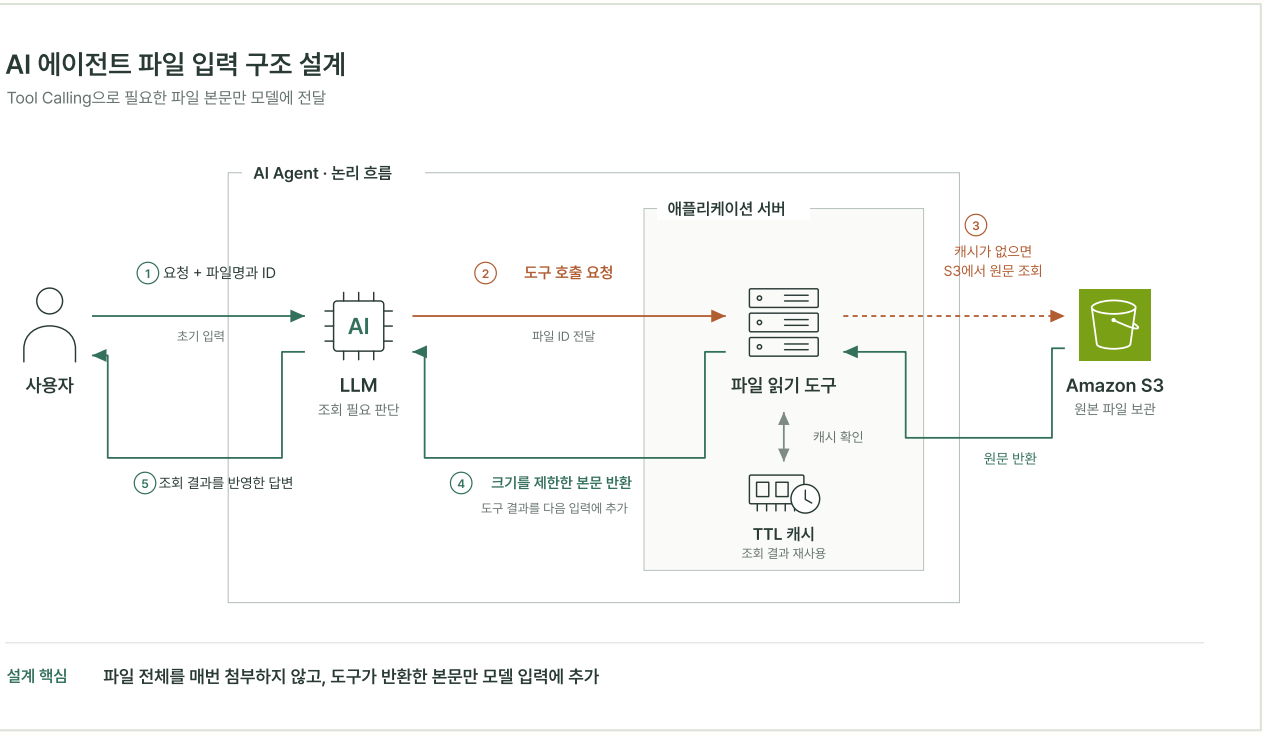
파일이 필요하지 않은 요청에도 전체 본문이 모델 입력에 포함됐습니다. 같은 내용을 반복 처리하면서 컨텍스트가 커졌습니다.

TO-BE

도구 기반 파일 조회

원문은 S3에 보관하고 파일 ID는 DB에서 관리했습니다. 모델에는 파일명과 ID만 전달했습니다. 본문이 필요할 때 파일 읽기 도구를 호출하도록 구성했습니다.

- TTL 캐시로 조회 결과를 재사용해 S3 반복 접근 감소



결과 파일 전체를 매번 모델에 전달하던 작업을 없애고, 실제로 읽은 범위만 다음 입력에 포함하도록 변경했습니다.

Argo CD Agent

2025.07 — 2025.12 Go 코드 개선 · Kubernetes 매니페스트 수정

2025 오픈소스 컨트리뷰션 아카데미 · 한국IT비즈니스진흥협회장상 수상

쿠버네티스 리소스 조회를 로컬 캐시로 개선

Go | Kubernetes | Argo CD

오픈소스 컨트리뷰션 아카데미에서 GitOps 기반 배포를 지원하는 Argo CD Agent에 기여했습니다.

기존 이슈: 반복적인 API 서버 호출

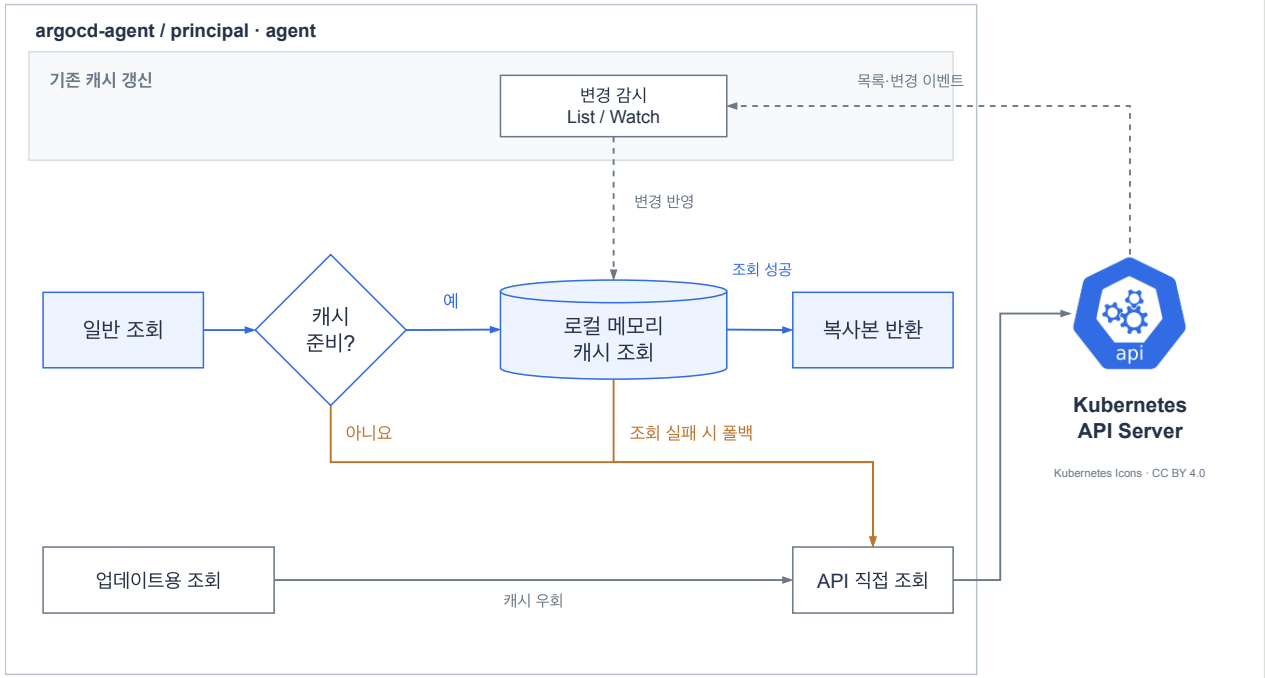
Application을 조회할 때마다 Kubernetes API 서버를 호출해, 반복 조회 과정에서 지연과 서버 처리 부담이 발생했습니다.

해결 방안: 로컬 캐시 적용과 조회 경로 분리

일반 조회에는 기존 로컬 메모리 캐시를 활용하고, 최신 정보가 필요한 업데이트용 조회는 API 서버 직접 호출을 유지했습니다.

- 캐시가 준비되지 않았거나 대상 조회가 실패하면 API 서버로 폴백
- 업데이트 재시도에서 오래된 정보를 읽지 않도록 조회 경로 분리
- 호출자가 캐시의 공유 객체를 변경하지 않도록 복사본 반환
- 수정한 코드의 테스트와 리뷰를 거쳐 공식 프로젝트에 병합

Application 조회 · 캐시와 API 서버



프로젝트 공식 아키텍처 ↗

기여 성과

- **100개 Application 조회 시간: 약 19초에서 0.4밀리초로 단축**
- **같은 실험의 API 호출 횟수: 300회에서 200회로 감소**
- 코드와 매니페스트 개선: 공식 프로젝트에 PR 5개 병합
- 오픈소스 기여 활동으로 한국IT비즈니스진흥협회장상 수상



표시된 두 구간의 누적 요청 증가량은 각각 300회와 200회입니다.

공식 프로젝트에 병합한 PR 5개

조회 성능 개선 외에도 API 연동 테스트와 네트워크 정책, 테스트 환경 설정을 보완했습니다. 아래 PR은 모두 공식 저장소에 병합됐습니다.

#525 Application 조회에 로컬 캐시 적용

일반 조회는 로컬 캐시를 사용하고, 최신 정보가 필요한 업데이트용 조회는 API 서버를 직접 호출하도록 분리했습니다.

#549 Argo CD API 연동 E2E 테스트 추가

API로 Application을 생성·조회·삭제하는 시나리오를 추가하고, 인증과 요청 서명을 처리하는 테스트 클라이언트를 보완했습니다.

#574 Redis 통신을 위한 NetworkPolicy 보완

Agent와 Principal이 Redis에 접근할 수 있도록 Helm 템플릿과 Kustomize의 네트워크 정책을 보완했습니다.

#603 캐시 동기화 대기시간 설정 추가

리소스 정보가 캐시에 반영될 때까지 기다리는 시간을 설정하도록 옵션을 추가하고, 기본값과 설정 동작을 단위 테스트로 확인했습니다.

#607 테스트용 Redis Proxy 비활성화 옵션 추가

테스트에서 불필요한 Redis Proxy를 끄고 Principal 서버를 실행할 수 있도록 옵션과 단위 테스트를 추가했습니다.

연구

VLM·객체 탐지 모델의 SLO 스케줄링

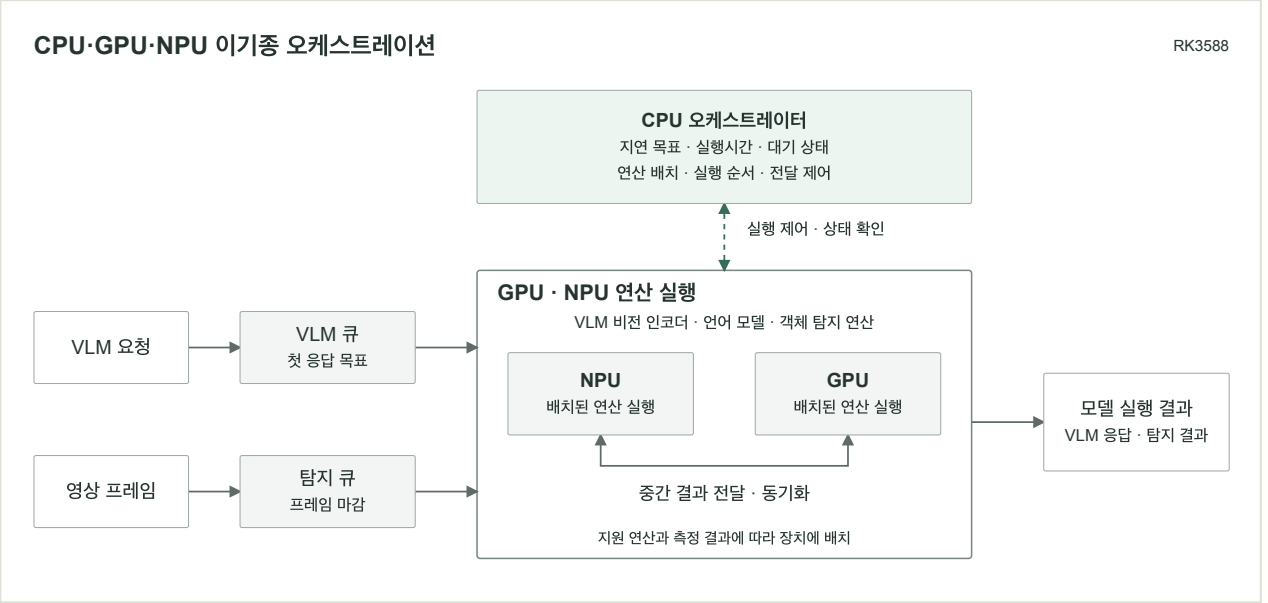
연구 진행 중 이화여자대학교 AI컴퓨팅 플랫폼 연구실 · 학부연구생

VLM과 YOLO를 함께 서빙하는 엣지 환경에서 NPU 자원 경쟁을 연구합니다. 모델별 코어 배분을 조정해 VLM의 응답시간 목표를 유지하면서 YOLO의 SLO 달성률을 20%p 개선했습니다.

연구 배경

영상의 객체 탐지와 이미지에 대한 질의응답을 한 엣지 장치에서 처리하는 환경을 다룹니다. CPU·GPU·NPU가 서로 다른 실행 특성을 갖는 환경에서 VLM과 객체 탐지 모델의 연산을 나누어 실행하는 이기종 오케스트레이션을 연구합니다.

CPU는 요청 관리와 실행 제어를 맡고, GPU와 NPU는 장치가 지원하는 모델 연산을 분담하는 구조를 대상으로 합니다. 각 장치에 연산을 배치하는 것뿐 아니라, 앞선 연산의 **중간 결과를 다음 장치에 넘기고 완료를 기다리는 과정까지 조정**해야 합니다.



홈페이지 ↗

비교 실험 결과

NPU 3개 코어를 사용하는 기존 교대 방식과 코어를 나눠 동시 실행하는 방식을 비교했습니다. VLM은 첫 토큰 생성까지의 지연시간, YOLO는 프레임별 응답시간으로 평가했습니다. VLM의 응답시간 목표를 유지하면서 YOLO의 SLO 달성률을 20%p 개선했습니다.

실시간 결제 서비스

AWS 인프라 설계 및 CI/CD 배포 자동화

2025.03 — 2025.12 백엔드 개발 · 클라우드 인프라 구축

AWS | Docker | CI/CD | Amazon SQS

프로젝트 개요

AWS·GCP 기반 멀티클라우드 환경에서 AWS 인프라 구축과 배포 자동화를 담당했습니다. 제한된 자원을 효율적으로 활용하면서 서비스 가용성을 확보하고, 요청량 변화에 대응할 수 있는 확장성도 함께 고려했습니다.

도입 배경	해결 방안
운영 EC2에서 빌드 중 메모리 부족 발생	CodeBuild로 빌드 환경 분리, CI/CD와 블루그린 배포 적용
신속한 결제 서비스 복구가 필요하나 운영자의 상시 대응은 어려움	Multi-AZ와 자동 장애 조치로 수동 복구 대기 최소화
교내 서비스 확대 시 사전에 확정하기 어려운 요청량에 대응 필요	EC2 Auto Scaling으로 처리 용량 조정
외부에 서비스를 제공하면서 데이터베이스의 직접 접근은 제한	프라이빗 서브넷과 보안 그룹으로 접근 제어
적재 작업 중단 시에도 수집 결과를 보관하고 재처리할 필요	Amazon SQS로 수집과 적재 디커플링
외부 푸시 서비스의 지연이 결제 응답을 늦추지 않도록 분리 필요	Redis Streams로 푸시 알림 비동기 처리

성과

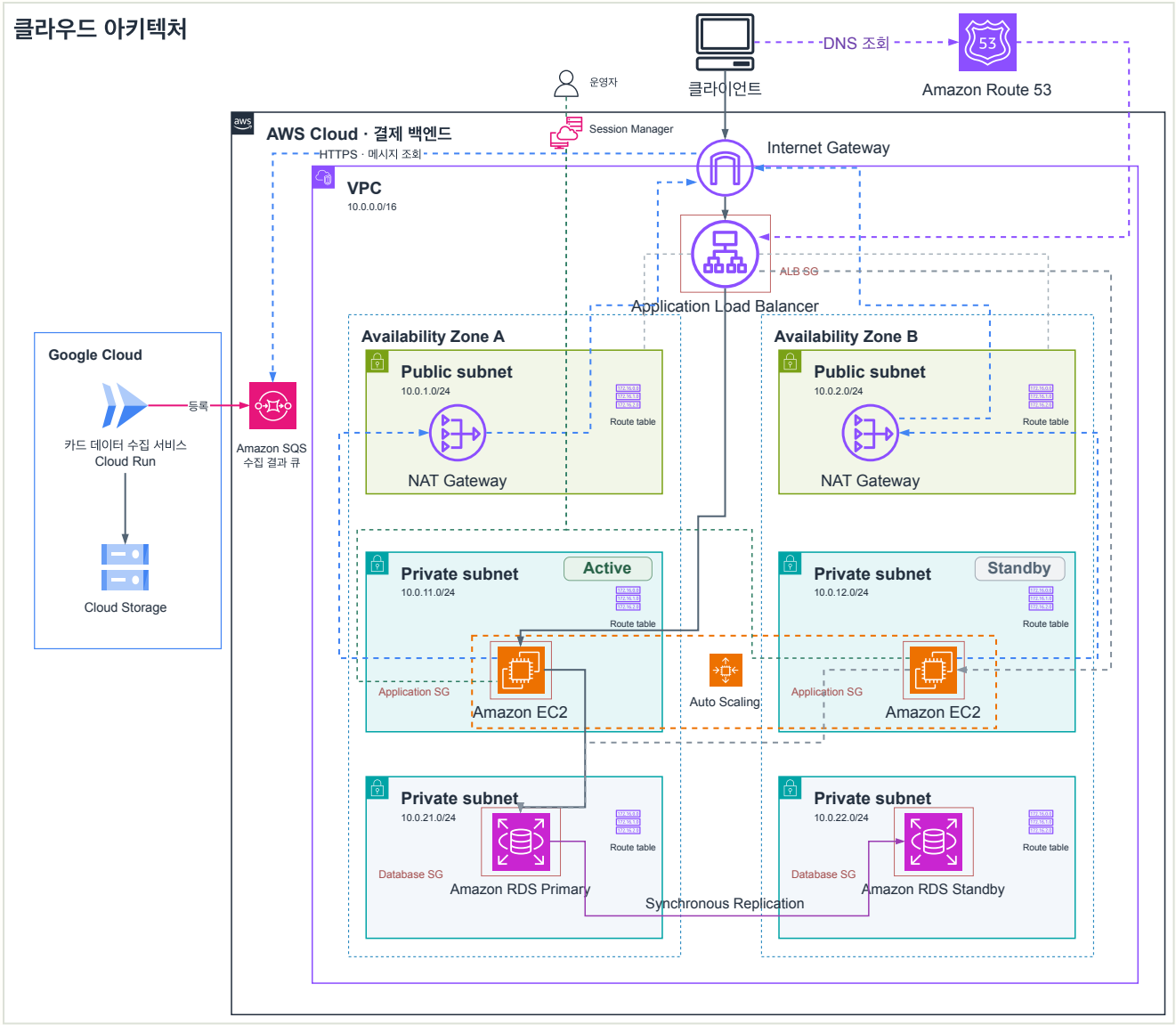
결제 서비스 10분 이내 복구 확인

단일 EC2 장애와 Primary DB 장애를 각각 발생시킨 복구 테스트에서, 결제 요청 처리가 10분 이내 재개되는 것을 확인했습니다.

배포 시간 30분 → 10분 단축

CI/CD 파이프라인으로 빌드와 배포 과정을 자동화했습니다.

클라우드 아키텍처



핵심 설계

다중 AZ 배포(Multi-AZ)

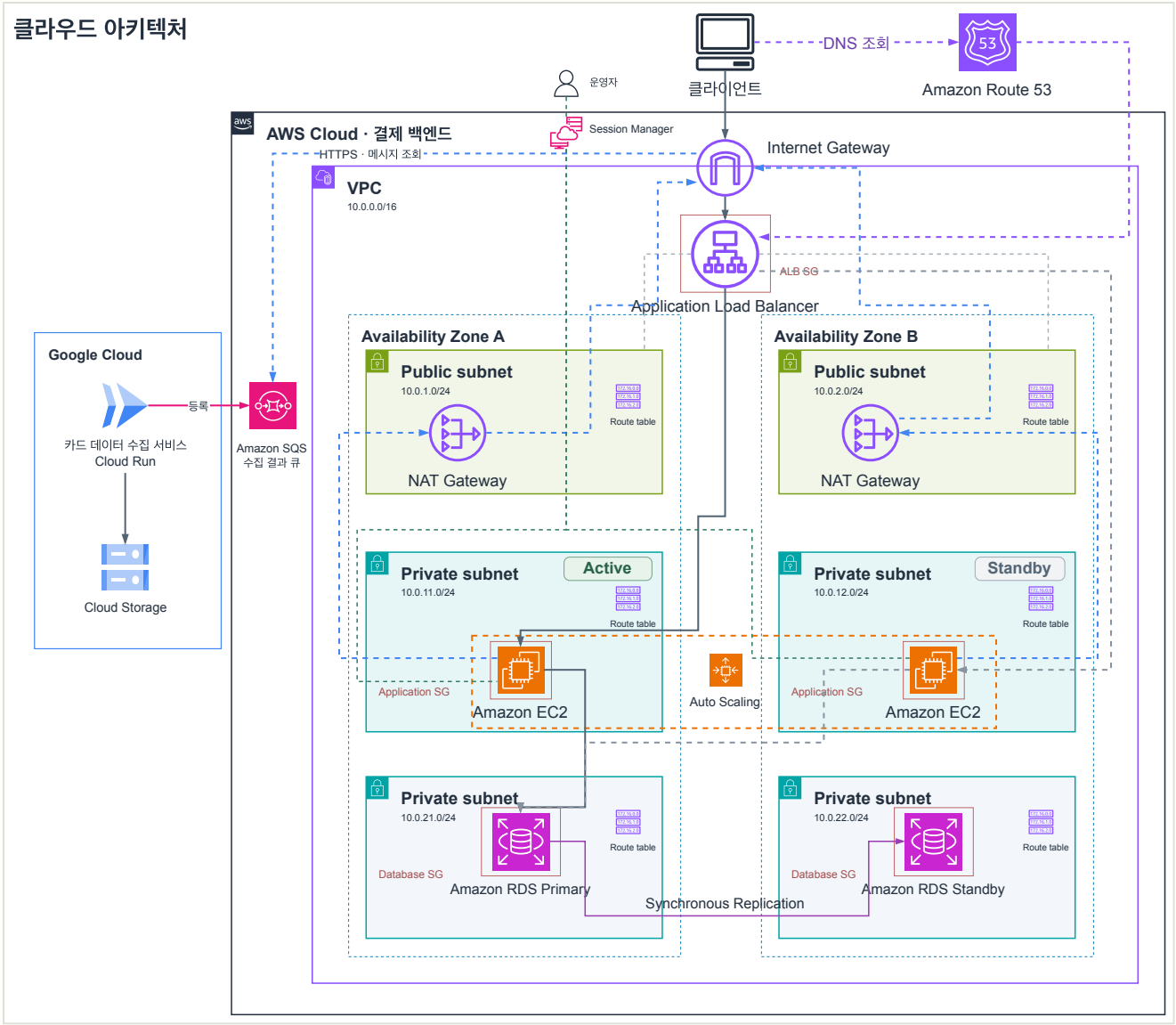
> 결과 **단일 EC2 장애와 Primary DB 장애를 각각 발생시킨 복구 테스트에서 결제 요청 처리가 10분 이내 재개되는 것을 확인했습니다.**

결제 서비스는 장애 발생 시 신속한 복구가 필요했지만, 운영자가 서비스 이용 시간 동안 상시 대응하기는 어려웠습니다. 수동 조치를 기다리는 시간을 줄이기 위해 고가용성(HA) 구조를 설계했습니다.

EC2를 서로 다른 가용영역에 배치하고, ALB 헬스 체크로 인스턴스 상태를 확인하도록 구성했습니다. RDS는 Multi-AZ DB 인스턴스 배포를 적용해 다른 가용영역의 Standby DB에 데이터를 동기 복제하고, Primary DB 장애 시 자동 장애 조치(Failover)를 수행하도록 했습니다.

요청량에 따른 자동 확장

학과 단위 서비스가 교내로 확대되는 상황을 고려해, 요청량 변화에 따라 처리 용량을 조정할 수 있도록 설계했습니다. 제한된 예산을 고려하면서 필요한 용량을 확보할 수 있도록 EC2 Auto Scaling 기반의 확장 구조를 구성했습니다.



핵심 설계 (계속)

네트워크 접근 제어

외부 요청은 퍼블릭 서브넷의 ALB에서 받아 프라이빗 서브넷의 애플리케이션으로 전달합니다. RDS도 프라이빗 서브넷에 배치하고, 보안 그룹으로 애플리케이션에서 필요한 통신만 허용해 인터넷에 직접 노출되지 않도록 설계했습니다.

Amazon SQS 기반 디커플링

수집 서비스와 DB 적재 작업 사이에 Amazon SQS를 두어 두 작업을 디커플링하는 구조를 설계했습니다. 적재 작업이 일시 중단되어도 수집 결과를 큐에 보관하고, 처리 재개 후 RDS에 반영하도록 구성합니다.

DB 반영에 성공한 메시지만 큐에서 삭제하고, 재전달된 메시지는 카드별 수집 ID를 기준으로 중복 반영 여부를 확인하도록 설계합니다.

CI/CD 파이프라인과 무중단 배포

빌드 환경 분리

- 문제: 운영 EC2에서 빌드 중 메모리 부족 발생
- 해결: **한정된 운영 자원을 서비스 실행에 집중**할 수 있도록 빌드를 CodeBuild로 분리하고, EC2에서는 완성된 컨테이너 이미지만 실행

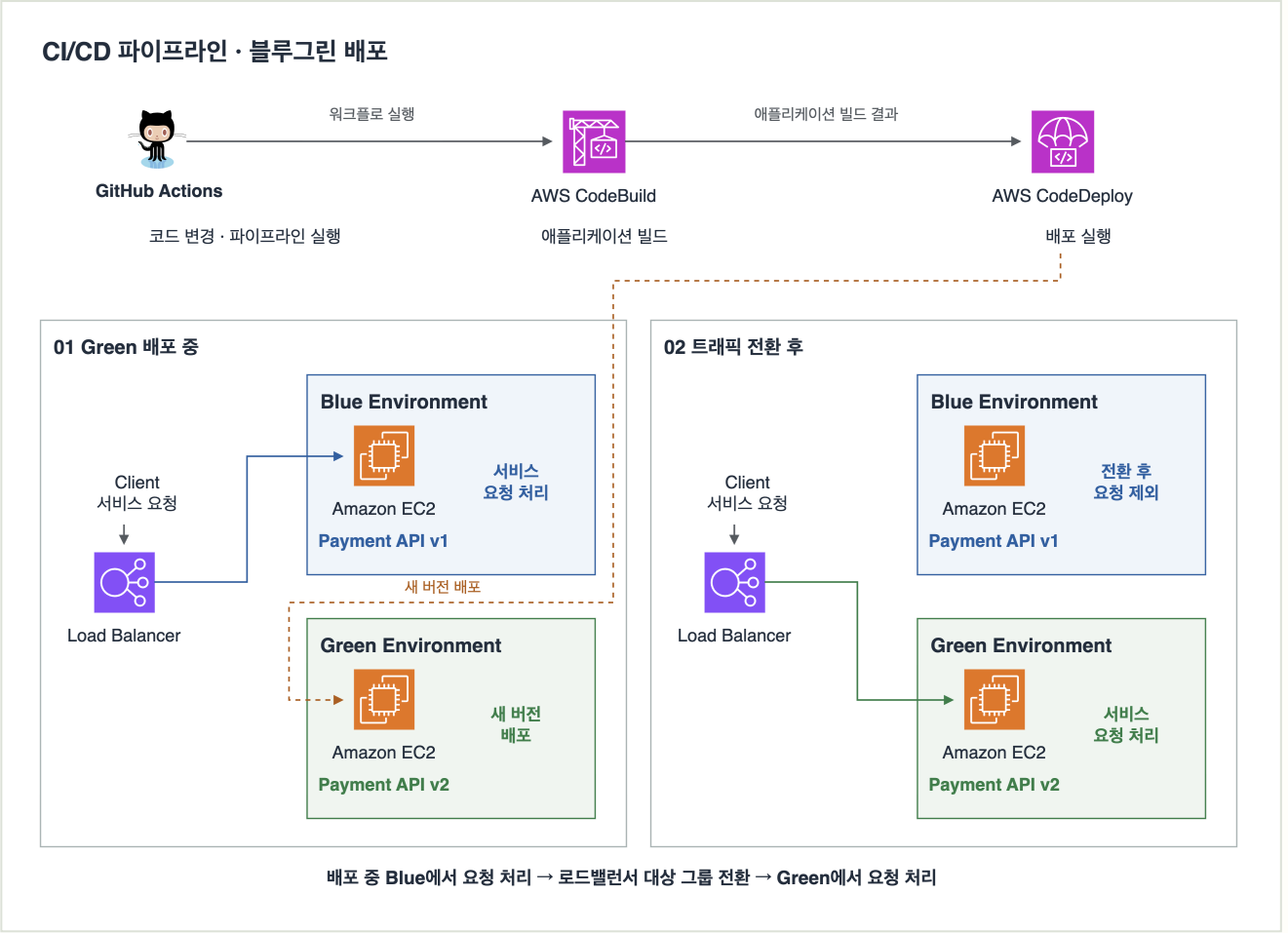
빌드와 배포 자동화

GitHub Actions와 CodeBuild, CodeDeploy를 연동해 수동 배포를 자동화했습니다. 서비스별 Dockerfile로 이미지를 정의하고 Docker Compose로 실행 설정을 관리했습니다.

블루그린 배포

배포로 인한 서비스 중단시간(다운타임)을 최소화하기 위해 블루그린 무중단 배포를 적용했습니다. Green 환경에 새 버전을 배포하고, 헬스 체크 통과를 확인한 뒤 로드밸런서의 트래픽을 전환했습니다.

> **결과** 배포 시간 30분 → 10분



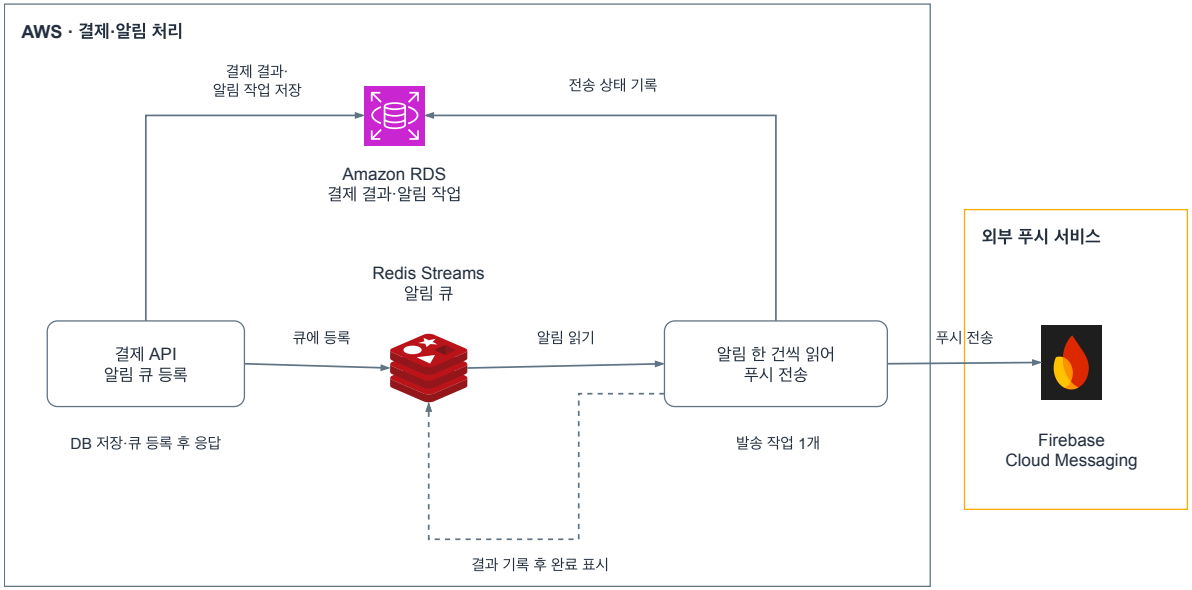
Redis를 이용한 푸시 알림 비동기 처리

푸시 전송을 기다리지 않고 결제 완료 응답

결제 서비스는 결제 결과와 알림 작업을 하나의 DB 트랜잭션으로 저장한 뒤 응답합니다. 푸시 전송은 Redis Streams를 통해 비동기로 처리해, 외부 푸시 서비스의 지연이 결제 응답에 영향을 주지 않도록 구성했습니다.

백엔드가 알림 작업을 Redis Streams에 등록하면, 발송 작업이 알림을 읽어 Firebase Cloud Messaging으로 전송합니다. 전송 결과를 DB에 기록하고, 전송에 성공한 알림은 큐에 처리 완료로 표시합니다.

결제 결과 저장과 푸시 알림 처리



Kubernetes Operator

운영 자동화 · 주간 리포트 작성 시간 150분 → 30분

스터디 프로젝트 Operator 개발 · 배포 상태 모니터링 Kubernetes Operator Loki

서비스 중단과 수동 점검 문제

서비스 중단이 반복됐지만 애플리케이션 상태와 관련 로그를 수동으로 확인해야 해, 장애 파악과 대응이 늦어졌습니다. 이를 개선하기 위해 Kubernetes Operator를 개발해 배포 상태 점검과 Slack 알림을 자동화했습니다.

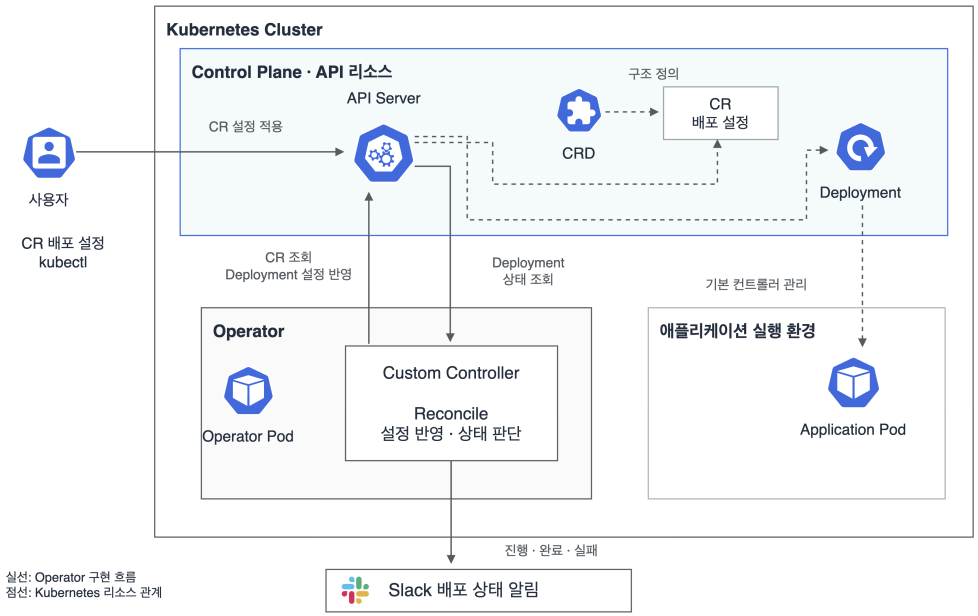
CRD와 Reconcile을 활용한 리소스 관리

- CRD로 배포 설정의 구조 정의
- Reconcile에서 Custom Resource를 읽어 Deployment 구성
- 실행 상태와 함께 서비스 준비 상태를 확인해 Slack으로 알림
- 실제 상태 변화와 알림 수신 내역을 대조해 동작 확인

AI 로그 분석과 장애 대응 자동화 체계

LLM에 Loki API를 연결하고, 애플리케이션의 호출 관계와 자원 사용 관계를 지식 그래프로 구축했습니다. 연결된 애플리케이션의 로그와 자원 상태까지 분석해 주간 리포트 작성 시간을 150분에서 30분으로 줄였습니다. 모의 장애에서 원인 분석 성공률 90%를 확인하고, 분석 지침과 도구를 팀에 공유했습니다.

Kubernetes Operator 아키텍처

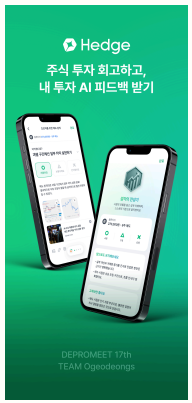


Operator는 API 서버를 통해 CR과 Deployment를 관리하고, 배포 상태를 Slack으로 전달합니다.

프로젝트

햇제, AI 주식 회고 애플리케이션 앱스토어 출시

2025.08 — 2026.02 디프만 · AI Agent / Backend · Java / Spring Boot / Python 팀 12명



멀티 에이전트 설계

매매 기록과 시장 데이터로 개인화된 투자 회고를 생성하는 서비스입니다. 단일 에이전트 구조를 역할별로 분리하고, 중앙 에이전트가 요청에 따라 시장 데이터와 사용자 기록을 담당하는 에이전트를 호출하도록 설계했습니다.

AI 응답 품질 개선

Sentence-BERT와 ChromaDB로 회고에 필요한 근거를 검색하고 Cross-Encoder reranking을 적용했습니다. Langfuse trace와 LLM-as-a-Judge 평가로 AI 응답 품질 지표를 30%에서 70%로 개선했습니다.

사용자 데이터 접근 제어

JWT에서 검증한 사용자 정보를 기준으로 거래 데이터의 접근 범위를 제한하고, 에이전트가 다른 사용자의 거래 내역에 접근하지 못하도록 설계했습니다.

홈페이지 ↗

뮤사이, 악기 거래 플랫폼

세아해암학술장학재단 커리어챌린지 장학금 300만 원

2024.09 — 2025.01 세오스 · Backend · Java / Spring Boot 팀 8명



현장 요구사항 반영

악기 상가를 직접 방문해 점주의 거래 방식과 재고 관리 절차를 파악했습니다. 고객용 예약과 점주용 관리 기능을 API 명세와 DB 스키마로 구체화했습니다.

예약·재고 정합성

예약 저장과 재고 차감을 하나의 트랜잭션으로 처리하고, 비관적 락으로 동시 예약을 제어했습니다. 변경 이력을 남겨 재고 수량과 거래 상태를 추적하도록 했습니다.

검색과 오류 추적

데이터베이스 인덱스를 적용해 검색 성능을 개선했습니다. Sentry로 서버 예외 알림과 오류 추적을 구성했습니다.

홈페이지 ↗

학력 및 주요 이력

학력

이화여자대학교 컴퓨터공학전공

2023 — 2027 · GPA 3.97 / 4.5

자격증

AWS Solutions Architect – Associate

AWS 공인 자격

배지 확인 ↗

SQLD

SQL 개발자

수상

한국IT비즈니스진흥협회장상

2025 오픈소스 컨트리뷰션 아카데미

세아해암학술장학재단 커리어챌린지

뮤사이 프로젝트 · 장학금 300만 원

개발 활동

AWS Cloud Club

이화여대 Captain · 공동 회장

10개 대학 · 200명 이상 규모의 연합동아리 운영

배지 확인 ↗

GDGoC Ewha

서버 Core Member

Spring Boot 스터디 개설

디프만

햇제 프로젝트 · AI 에이전트·백엔드 개발

세오스

뮤사이 프로젝트 · 백엔드 개발

프로젝트에 사용한 기술

Backend	Java, Spring Boot, JPA, QueryDSL, Python, Go
Data	MySQL, Redis
Infrastructure	AWS, GCP, Docker, Kubernetes, Argo CD
Delivery & Ops	GitHub Actions, CodeBuild, CodeDeploy, Loki, Sentry